
Introduction to Python and Data Analysis

Rennan Marujo





Python Environment



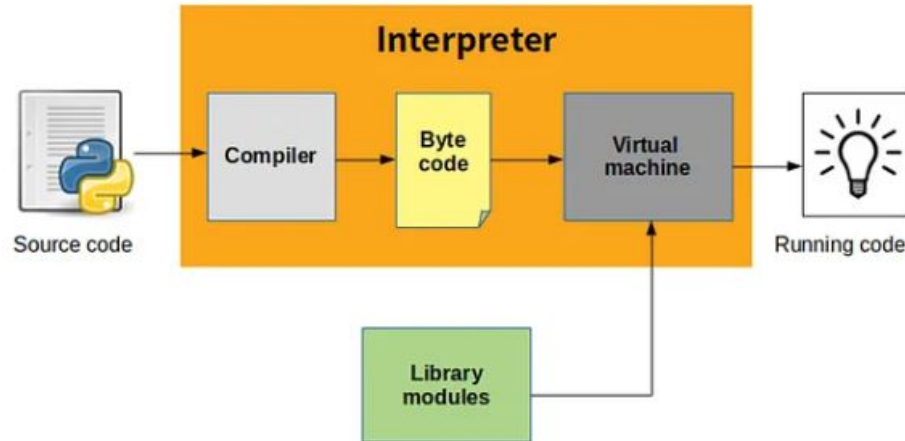
Guido Van Rossum

How does python work?

Compiler



Interpreter



Conda

```
wget https://repo.anaconda.com/archive/Anaconda3-2025.12-2-Linux-x86\_64.sh  
chmod +x Anaconda3-2025.12-2-Linux-x86\_64.sh  
./Anaconda3-2025.12-2-Linux-x86\_64.sh
```

yes

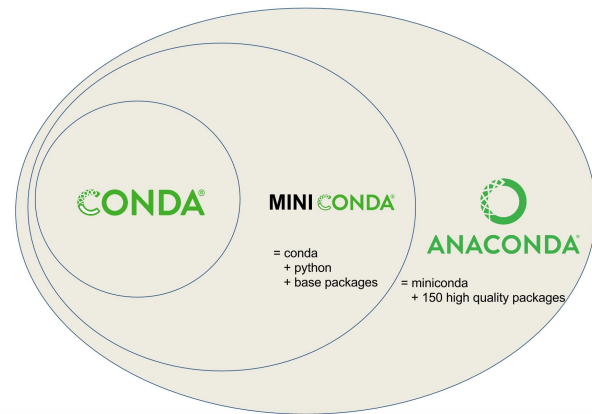
Enter

yes

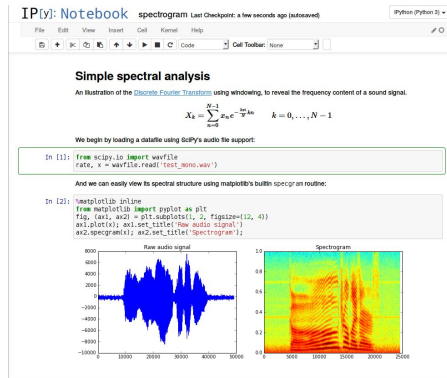
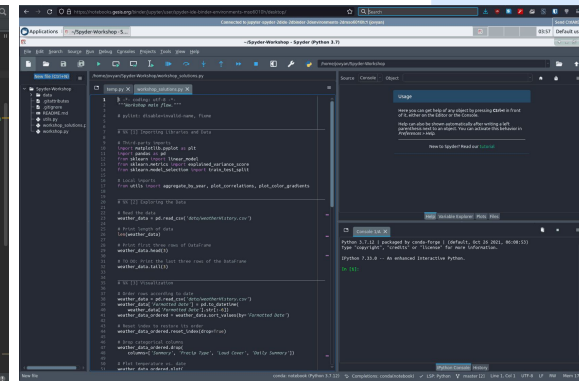
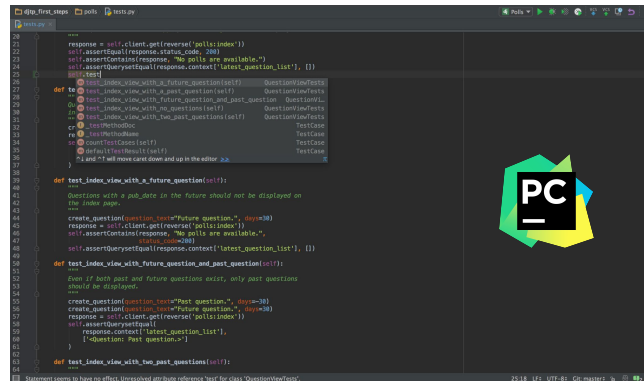
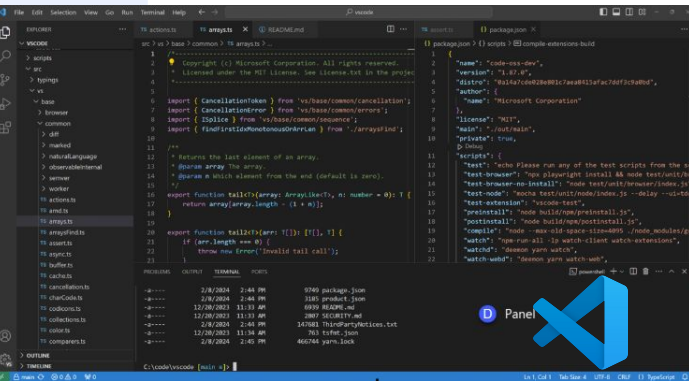
```
conda activate
```

```
conda info --envs
```

```
conda create -n myenv python=3.14
```



Integrated Development Environment (IDE)





Python Review

Python introduction





SER-347/CAP-419

Pesquisar documentos

Introdução à Programação com Dados Geoespaciais

VISÃO GERAL DO CURSO:

Visão Geral do Curso

AULAS:

1. Instalando e Configurando o Ambiente de Programação
2. Introdução à Programação com a Linguagem Python

- 2.1. Introdução
- 2.2. Primeiro Programa em Python
- 2.3. Tipos de Dados
- 2.4. Tipos Numéricos
- 2.5. Expressões
- 2.6. Chamada de Funções
- 2.7. Variáveis
- 2.8. Comentários
- 2.9. Tipo Lógico
- 2.10. Operadores Relacionais
- 2.11. Expressões Lógicas
- 2.12. Estruturas Condicionais
- 2.13. Estruturas de Repetição

🏠 / 2. Introdução à Programação com a Linguagem Python

◀ Anterior

Próximo ▶

2. Introdução à Programação com a Linguagem Python

Um **programa** é formado por uma **seqüência de instruções** que diz ao computador como realizar uma determinada tarefa. As instruções podem dizer como o programa deve interagir com o usuário, como ele deve utilizar o hardware, e como operar os dados para realizar o que chamamos de **computações**. Quando um computador segue as instruções de um programa, dizemos que ele está **executando o programa**.

A atividade de **programar** consiste na habilidade de expressar as computações que dizem ao computador como resolver um determinado problema. Uma das ferramentas que permitem expressar essas computações são as **Linguagens de Programação**. O **Trecho de Código 2.1** exemplifica um programa escrito na linguagem Python, composto por uma seqüência de instruções encadeadas de forma a produzir o gráfico com a série temporal mostrado na Figura 2.1.

Trecho de Código 2.1 - Recuperando uma série temporal.

```
1 import matplotlib.pyplot as plt
2 from wtss import wtss
3
4 w = wtss('http://www.esensing.dpi.inpe.br/')
5
6 ts = w.time_series(coverage='M0013Q1', attributes=('red', 'nir'),
7               latitude=-12.0, longitude=-54.0,
8               start_date='2001-01-01', end_date='2001-12-31')
9
10 fig, ax = plt.subplots()
11
12 plt.title('Time Series M0013Q1', fontsize=24)
13
14 plt.xlabel('Date', fontsize=16)
15 plt.ylabel('Surface Reflectance', fontsize=16)
16
17 ax.plot(ts.timeline, ts['red'], color='red', ls='-', marker='o', label='red')
18 ax.plot(ts.timeline, ts['nir'], color='purple', ls='-', marker='o', label='nir')
19
20 plt.legend()
21
22 plt.grid(b=True, color='gray', linestyle='--', linewidth=0.5);
```

Numerical Data types

Numeric (int, float and complex)

Operador	Nome	Expressão	Valor
<code>+</code>	soma	<code>5 + 8</code>	13
<code>-</code>	subtração	<code>3 - 2</code>	1
<code>*</code>	multiplicação	<code>3 * 4</code>	12
<code>/</code>	divisão	<code>6 / 4</code>	1.5
<code>%</code>	resto da divisão	<code>6 % 4</code>	2
<code>**</code>	potenciação	<code>2 ** 4</code>	16

```
type(5)
```

Last executed at 2026-06-05 22:38:20 in 9ms

int

```
type(2.5)
```

Last executed at 2026-06-05 22:38:26 in 9ms

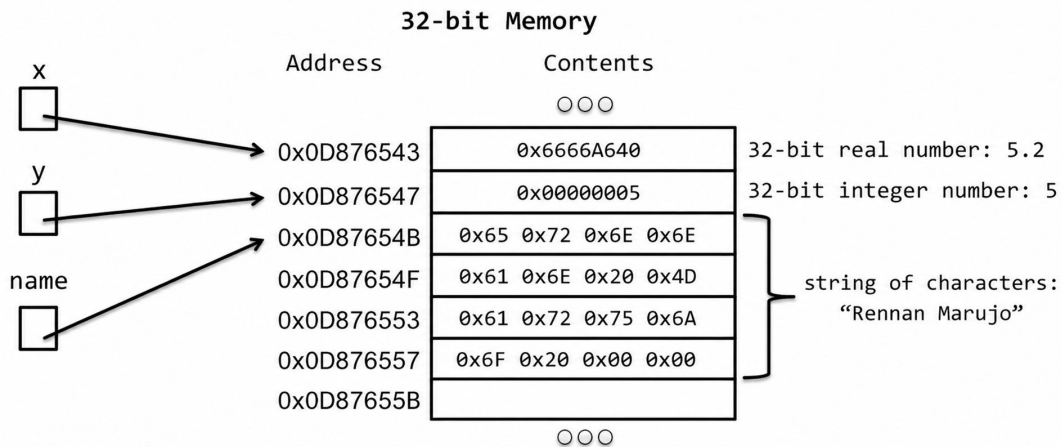
float

```
type(7+2j)
```

Last executed at 2026-06-05 22:38:33 in 9ms

complex

Variables



```
### This is a comment
```

```
var = None  
type(var)
```

```
Last executed at 2026-06-05 22:37:47 in 13ms
```

```
NoneType
```

Logical types

```
type(True)
```

```
Last executed at 2026-06-05 22:38:59 in 8ms
```

```
bool
```

```
type(False)
```

```
Last executed at 2026-06-05 22:39:02 in 8ms
```

```
bool
```

Tabela 2.7 - Tabela verdade do operador `and`.

p	q	$p \wedge q$
True	True	True
True	False	False
False	True	False
False	False	False

Tabela 2.8 - Tabela verdade do operador `or`.

p	q	$p \vee q$
True	True	True
True	False	True
False	True	True
False	False	False

Tabela 2.9 - Tabela verdade do operador `not`.

p	$\neg p$
True	False
False	True

String type

```
name = "Data"  
type(name)  
Last executed at 2026-06-05 22:42:50 in 8ms
```

str

```
name = name + " Science"  
name  
Last executed at 2026-06-05 22:43:01 in 8ms
```

'Data Science'

```
name * 3  
Last executed at 2026-06-05 22:43:14 in 9ms
```

'Data ScienceData ScienceData Science'

```
"Data" in name  
Last executed at 2026-06-05 22:43:28 in 8ms
```

True

```
"science" not in "name"  
Last executed at 2026-06-05 22:43:56 in 8ms
```

True

```
len(name)  
Last executed at 2026-06-05 22:44:03 in 8ms
```

12

```
name[5]  
Last executed at 2026-06-05 22:44:15 in 7ms
```

'S'

```
name[5:]  
Last executed at 2026-06-05 22:44:23 in 9ms
```

'Science'

```
name.find("Sci")  
Last executed at 2026-06-05 22:44:43 in 7ms
```

5

```
name.split(" ")  
Last executed at 2026-06-05 22:44:58 in 9ms
```

['Data', 'Science']

```
name.replace(" ", "_")  
Last executed at 2026-06-05 22:45:13 in 8ms
```

'Data_Science'

```
name.upper()  
Last executed at 2026-06-05 22:45:17 in 8ms
```

'DATA SCIENCE'

```
name.lower()  
Last executed at 2026-06-05 22:45:23 in 8ms
```

'data science'

```
name.isupper()  
Last executed at 2026-06-05 22:45:29 in 8ms
```

False

```
name.islower()  
Last executed at 2026-06-05 22:45:33 in 8ms
```

False

Operators

```
1 < 2
```

Last executed at 2026-06-05 22:48:06 in 8ms

True

```
a = 2
```

```
b = 3
```

```
b > a
```

Last executed at 2026-06-05 22:48:18 in 7ms

True

```
c = 2
```

```
a >= c
```

Last executed at 2026-06-05 22:48:50 in 7ms

True

```
b <= a
```

Last executed at 2026-06-05 22:48:56 in 8ms

False

```
a == c
```

Last executed at 2026-06-05 22:49:08 in 8ms

True

```
a != c
```

Last executed at 2026-06-05 22:49:11 in 9ms

False

Condicionais

```
a = 1
if a == 1:
    print("It's equal to one!")
```

Last executed at 2026-06-05 22:50:58 in 8ms

It's equal to one!

```
a = 2
if a == 1:
    print("It's equal to one!")
else:
    print("It's not equal to one!")
```

Last executed at 2026-06-05 22:51:28 in 7ms

It's not equal to one!

```
a = 2
if a == 1:
    print("It's equal to one!")
if a == 2:
    print("It's equal to two!")
else:
    print("It's not equal to one!")
```

Last executed at 2026-06-05 22:51:53 in 7ms

It's equal to two!

```
a = 2
if a == 1:
    print("It's equal to one!")
else:
    if a == 2:
        print("It's equal to two!")
        print("It's not equal to one!")
```

Last executed at 2026-06-05 22:52:25 in 8ms

It's equal to two!

It's not equal to one!

Loops

```
for i in range(5):  
    print(i)
```

Last executed at 2026-06-05 22:53:30 in 7ms

0
1
2
3
4

```
for i in range(5):  
    if i == 3:  
        break  
    print(i)
```

Last executed at 2026-06-05 22:53:52 in 7ms

0
1
2

```
for i in range(5):  
    if i < 3:  
        continue  
    print(i)
```

Last executed at 2026-06-05 22:54:05 in 6ms

3
4

```
i = 0  
while i < 5:  
    print(i)  
    i += 1 # i = i + 1
```

Last executed at 2026-06-05 22:55:34 in 10ms

0
1
2
3
4

```
i = 5  
while i <= 5:  
    try:  
        print(10/i)  
        i -= 1 # i = i - 1  
    except:  
        print(f"Error when i was {i}!")  
        break
```

Last executed at 2026-06-05 22:57:14 in 10ms

2.0
2.5
3.3333333333333335
5.0
10.0
Error when i was 0!

Functions and scope

```
def subtract2(x):  
    value = x - 2  
    return value
```

```
subtract2(10)
```

Last executed at 2026-06-05 22:59:53 in 10ms

8

```
value
```

Last executed at 2026-06-05 22:59:59 in 26ms

```
-----  
NameError  
Cell In[53], line 1  
----> 1 value
```

Traceback (most recent call last)

```
NameError: name 'value' is not defined
```

Class

```
class Person():  
    def __init__(self, name, cpf):  
        self.name = name # attribute  
        self.cpf = cpf    # attribute  
    def say_name_and_cpf(self):  
        print(f"Hi, my name is {self.name} and my CPF is {self.cpf}")  
  
me = Person("Fulano", "123")  
print(me)  
print(me.name)  
print(me.cpf)  
print()  
print(me.say_name_and_cpf())
```

Last executed at 2026-06-05 23:04:41 in 11ms

```
<__main__.Person object at 0x7fb2e582ae90>  
Fulano  
123
```

```
Hi, my name is Fulano and my CPF is 123  
None
```

Lists

```
my_list = list()
other_list = []
my_list.append("Potato")
my_list.append("Garlick")
my_list.append("Potato")
my_list.append("Onion")
my_list
```

Last executed at 2026-06-05 23:22:04 in 10ms

```
['Potato', 'Garlick', 'Potato', 'Onion']
```

```
my_list.insert(1, "Cheese")
my_list
```

Last executed at 2026-06-05 23:22:04 in 8ms

```
['Potato', 'Cheese', 'Garlick', 'Potato', 'Onion']
```

```
my_list.extend("Onion")
my_list
```

Last executed at 2026-06-05 23:22:04 in 8ms

```
['Potato', 'Cheese', 'Garlick', 'Potato', 'Onion', 'O', 'n', 'i', 'o', 'n']
```

```
my_list = my_list[:5]
my_list
```

Last executed at 2026-06-05 23:22:05 in 7ms

```
['Potato', 'Cheese', 'Garlick', 'Potato', 'Onion']
```

```
my_list.extend(["Onion", "Milk", "Potato"]) # my_list = my_list + ["Onion", "Milk", "Potato"]
my_list
```

Last executed at 2026-06-05 23:22:06 in 9ms

```
['Potato', 'Cheese', 'Garlick', 'Potato', 'Onion', 'Onion', 'Milk', 'Potato']
```

```
my_list.remove("Milk")
my_list
```

Last executed at 2026-06-05 23:22:08 in 8ms

```
['Potato', 'Cheese', 'Garlick', 'Potato', 'Onion', 'Onion', 'Potato']
```

```
my_list.pop()
```

Last executed at 2026-06-05 23:22:09 in 7ms

```
'Potato'
```

```
my_list
```

Last executed at 2026-06-05 23:22:10 in 8ms

```
['Potato', 'Cheese', 'Garlick', 'Potato', 'Onion', 'Onion']
```

```
my_list.pop()
```

Last executed at 2026-06-05 23:22:11 in 8ms

```
'Onion'
```

Lists

```
[132]: my_list
```

Last executed at 2026-06-05 23:22:12 in 9ms

```
[132]: ['Potato', 'Cheese', 'Garlick', 'Potato', 'Onion']
```

```
[133]: del my_list[2]  
my_list
```

Last executed at 2026-06-05 23:22:13 in 8ms

```
[133]: ['Potato', 'Cheese', 'Potato', 'Onion']
```

```
[134]: my_list[-1]
```

Last executed at 2026-06-05 23:22:14 in 7ms

```
[134]: 'Onion'
```

```
[135]: my_list.index("Onion")
```

Last executed at 2026-06-05 23:22:17 in 8ms

```
[135]: 3
```

```
[136]: my_list.count("Potato")
```

Last executed at 2026-06-05 23:22:19 in 8ms

```
[136]: 2
```

```
my_list.count("Potato")
```

Last executed at 2026-06-05 23:22:19 in 8ms

```
2
```

```
my_list.sort()  
my_list
```

Last executed at 2026-06-05 23:22:21 in 8ms

```
['Cheese', 'Onion', 'Potato', 'Potato']
```

```
my_list.sort(reverse = True)  
my_list
```

Last executed at 2026-06-05 23:22:22 in 9ms

```
['Potato', 'Potato', 'Onion', 'Cheese']
```

```
len(my_list)
```

Last executed at 2026-06-05 23:22:23 in 9ms

```
4
```

```
min(my_list)
```

Last executed at 2026-06-05 23:22:24 in 8ms

```
'Cheese'
```

```
max(my_list)
```

Last executed at 2026-06-05 23:22:24 in 8ms

```
'Potato'
```

Lists

```
for i, value in enumerate(['a', 'b', 'c']):  
    print(f"{i}: {value}")
```

Last executed at 2026-06-05 23:22:29 in 7ms

```
0: a  
1: b  
2: c
```

```
names = ['Ana', 'João', 'Maria']  
ages = [25, 30, 28]  
combined = list(zip(names, ages))  
print(combined)
```

Last executed at 2026-06-05 23:22:30 in 8ms

```
[('Ana', 25), ('João', 30), ('Maria', 28)]
```



Sets and Tuples

```
set(my_list)
```

Last executed at 2026-06-05 23:22:26 in 9ms

```
{'Cheese', 'Onion', 'Potato'}
```

```
a = (1,2)
```

```
type(a)
```

Last executed at 2026-06-05 23:22:32 in 8ms

```
tuple
```

```
a
```

Last executed at 2026-06-05 23:22:33 in 8ms

```
(1, 2)
```

```
a[0]
```

Last executed at 2026-06-05 23:22:33 in 8ms

```
1
```

```
a[1]
```

Last executed at 2026-06-05 23:20:56 in 8ms

```
2
```

```
a[0]=3
```

Last executed at 2026-06-05 23:21:01 in 25ms

```
-----  
TypeError
```

Traceback (most recent call last)

```
Cell In[101], line 1
```

```
----> 1 a[0]=3
```

```
TypeError: 'tuple' object does not support item assignment
```



Dates

```
from datetime import datetime, date, time

# Current full date and time
current_datetime = datetime.now()
print(current_datetime)           # 2024-01-15 14:30:45.123456

# Current date only
today = date.today()
print(today)                     # 2024-01-15

# Current time only
current_time = datetime.now().time()
print(current_time)              # 14:30:45.123456
```

Last executed at 2026-06-05 23:36:21 in 10ms

2026-06-06 02:36:21.643253

2026-06-06

02:36:21.643770

```
print(type(current_datetime))
print(type(today))
print(type(current_time))
```

Last executed at 2026-06-05 23:36:21 in 7ms

```
<class 'datetime.datetime'>
<class 'datetime.date'>
<class 'datetime.time'>
```

```
from datetime import timedelta
```

```
tomorrow = today + timedelta(days=1)
yesterday = today - timedelta(days=1)
next_week = today + timedelta(weeks=1)
in_two_hours = current_datetime + timedelta(hours=2)
```

Last executed at 2026-06-05 23:37:13 in 8ms

Dictionary

a

Last executed at 2026-06-05 23:31:26 in 10ms

```
{'type': 'Feature',
 'id': 'LC09_L2SP_001057_20260602_20260603_02_T1',
 'collection': 'landsat-2',
 'stac_version': '1.0.0',
 'stac_extensions': ['https://stac-extensions.github.io/eo/v1.0.0/schema.json'],
 'geometry': {'type': 'Polygon',
 'coordinates': [[[[-62.64791033185974, 3.2866548410188954],
 [-62.64693781140399, 5.384835568446034],
 [-64.69953808440432, 5.382567225872584],
 [-64.69485927159307, 3.2852728084611678],
 [-62.64791033185974, 3.2866548410188954]]]]},
 'links': [],
 'bbox': [-64.69953808440432,
 3.2852728084611678,
 -62.64693781140399,
 5.384835568446034]}
```

a['bbox']

Last executed at 2026-06-05 23:31:46 in 8ms

```
[-64.69953808440432, 3.2852728084611678, -62.64693781140399, 5.384835568446034]
```

a['geometry']['coordinates'][0][0]

Last executed at 2026-06-05 23:32:34 in 9ms

```
[-62.64791033185974, 3.2866548410188954]
```



JSON

JavaScript Object Notation (JSON)

JSON (JavaScript Object Notation) is a lightweight data-interchange format. It is easy for humans to read and write. It is easy for machines to parse and generate. It is based on a subset of the JavaScript Programming Language Standard ECMA-262 3rd Edition – December 1999. JSON is a text format that is completely language independent but uses conventions that are familiar to programmers of the C-family of languages, including C, C++, C#, Java, JavaScript, Perl, Python, and many others. These properties make JSON an ideal data-interchange language.

JSON

```
import json

# Write dictionary to JSON file
with open('data.json', 'w', encoding='utf-8') as file:
    json.dump(dictionary, file, indent=4, ensure_ascii=False)

# Read dictionary from JSON file
with open('data.json', 'r', encoding='utf-8') as file:
    dictionary = json.load(file)
```

Last executed at 2026-06-05 23:39:29 in 10ms



NumPy



Numpy

```
import numpy as np
```

```
# Creating arrays
```

```
arr = np.array([1, 2, 3, 4, 5])
```

```
zeros = np.zeros((3, 3))      # 3x3 matrix of zeros
```

```
ones = np.ones((2, 4))        # 2x4 matrix of ones
```

```
# Vectorized operations
```

```
sum_arr = arr + 10             # [11, 12, 13, 14, 15]
```

```
mean = arr.mean()              # 3.0
```

```
matrix = np.array([[1, 2], [3, 4]])
```

```
result = matrix * 2            # [[2, 4], [6, 8]]
```

Last executed at 2026-06-05 23:52:11 in 549ms

```
# Basic: start, stop, step
```

```
arr1 = np.arange(5)            # [0, 1, 2, 3, 4]
```

```
arr2 = np.arange(2, 10)        # [2, 3, 4, 5, 6, 7, 8, 9]
```

```
arr3 = np.arange(2, 10, 2)     # [2, 4, 6, 8]
```

```
arr4 = np.arange(0, 1, 0.2)    # [0. , 0.2, 0.4, 0.6, 0.8]
```

```
import numpy as np
```

```
# 1D array with 12 elements
```

```
arr = np.arange(12) # [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]
```

```
# Reshape to 3x4 (3 rows, 4 columns)
```

```
arr_3x4 = arr.reshape(3, 4)
```

```
# [[ 0,  1,  2,  3],
```

```
#  [ 4,  5,  6,  7],
```

```
#  [ 8,  9, 10, 11]]
```

```
# Use -1 to infer dimension automatically
```

```
arr_auto = arr.reshape(4, -1) # 4x3 (calculates automatically)
```

```
print(arr_auto.shape) # (4, 3)
```

```
# Original array is not modified
```

```
print(arr.shape) # (12,) - unchanged
```

Numpy

```
import numpy.ma as ma

data = np.array([1, 2, -999, 4, -999, 6])

# Mask -999 values (treat as invalid)
masked = ma.masked_where(data == -999, data)
print(masked)
# [1, 2, --, 4, --, 6] (-- represents masked value)

# Operations ignore masked values
print(masked.mean()) # 3.25 (sum 1+2+4+6 = 13 / 4 = 3.25)
print(np.mean(data)) # -163.5 (includes -999!)

# Multiple conditions
data2 = np.array([10, -999, 20, -1, 30, 999])
masked2 = ma.masked_where((data2 == -999) | (data2 < 0) | (data2 > 100), data2)
print(masked2) # [10, --, 20, --, 30, --]

# Create masked array directly
valid_data = np.array([1, 2, 3, 4, 5])
mask = [False, True, False, True, False] # True = masked
masked_array = ma.masked_array(valid_data, mask=mask)
print(masked_array) # [1, --, 3, --, 5]

# Useful methods
print(masked_array.compressed()) # [1, 3, 5] - removes mask
print(masked_array.filled(-1)) # [1, -1, 3, -1, 5] - fills with value
print(masked_array.count()) # 3 (only unmasked)
```

```
import numpy as np

arr_2d = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])

# ravel() - flattens to 1D
flat = arr_2d.ravel()
print(flat) # [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Random
types



Pandas



<https://prog-geo.github.io/vetorial/pandas.html>

Pandas Series

```
# Creating a Series
s = pd.Series([10, 20, 30, 40, 50], name='numbers')
print(s)
# 0    10
# 1    20
# 2    30
# 3    40
# 4    50
# Name: numbers, dtype: int64

# Series from dictionary
s2 = pd.Series({'a': 1, 'b': 2, 'c': 3})
print(s2)
# a    1
# b    2
# c    3

# Access Series elements
print(s.iloc[2]) # 30
print(s.values)  # [10 20 30 40 50]
```



Pandas DataFrame

```
import pandas as pd

# Creating DataFrame from dict
df_dict = pd.DataFrame({
    'product': ['A', 'B', 'C'],
    'price': [100, 200, 300]
})
df_dict
```

Last executed at 2026-06-06 00:53:12 in 369ms

	product	price
0	A	100
1	B	200
2	C	300

```
# Creating DataFrame from List of Lists
df_list = pd.DataFrame([
    ['A', 100],
    ['B', 200],
    ['C', 300]
], columns=['product', 'price'])
df_list
```

Last executed at 2026-06-06 00:54:04 in 14ms

	product	price
0	A	100
1	B	200
2	C	300

```
print(df.head())
#   name age  city  salary
# 0  Alice  25   NYC   50000
# 1   Bob   30    LA   60000
# 2 Charlie  35   NYC   70000
# 3  Diana  28  Miami  55000
# 4   Eve   32    LA   65000
```

```
print(df.head(3)) # First 3 rows
#   name age city  salary
# 0  Alice  25   NYC   50000
# 1   Bob   30    LA   60000
# 2 Charlie  35   NYC   70000
```

```
print(df.tail())
#   name age  city  salary
# 1   Bob   30    LA   60000
# 2 Charlie  35   NYC   70000
# 3  Diana  28  Miami  55000
# 4   Eve   32    LA   65000
# 5  Frank  27   NYC   52000
```

```
print(df.tail(2)) # Last 2 rows
#   name age city  salary
# 4   Eve   32    LA   65000
# 5  Frank  27   NYC   52000
```

Pandas DataFrame

```
# DataFrame info
print(df_list.shape)      # (6, 4)
print()
print(df_list.info())     # Column types and memory
print()
print(df_list.describe()) # Statistics for numeric columns
```

Last executed at 2026-06-06 00:55:22 in 22ms

(3, 2)

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 3 entries, 0 to 2

Data columns (total 2 columns):

#	Column	Non-Null Count	Dtype
0	product	3 non-null	object
1	price	3 non-null	int64

dtypes: int64(1), object(1)

memory usage: 180.0+ bytes

None

	price
count	3.0
mean	200.0
std	100.0
min	100.0
25%	150.0
50%	200.0
75%	250.0
max	300.0

```
import pandas as pd
```

```
df = pd.DataFrame([
    'name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank'],
    'age': [25, 30, 35, 28, 32, 27],
    'city': ['NYC', 'LA', 'NYC', 'Miami', 'LA', 'NYC'],
    'salary': [50000, 60000, 70000, 55000, 65000, 52000]
])
```

1. Columns with 'a' in ANY position of the name

```
filtered_like = df.filter(like='a')
```

```
print("LIKE 'a':")
```

```
print(filtered_like.columns.tolist())
```

['name', 'age', 'salary'] ← city is NOT here!

```
print("\n" + "="*50 + "\n")
```

2. Columns that START with 'a' or 'c'

```
filtered_regex = df.filter(regex='^(a|c)')
```

```
print("REGEX '^(a|c)':")
```

```
print(filtered_regex.columns.tolist())
```

['age', 'city'] ← city appears because it starts with 'c'

Last executed at 2026-06-06 01:06:11 in 14ms

```
LIKE 'a':
```

```
['name', 'age', 'salary']
```

=====

```
REGEX '^(a|c)':
```

```
['age', 'city']
```

Pandas

```
print("Using iterrows():")
for idx, row in df.iterrows():
    print(f"Index: {idx}, Name: {row['name']}, Age: {row['age']}")
print()

# Modifying rows (does NOT modify original - need to use .at or .loc)
for idx, row in df.iterrows():
    df.at[idx, 'age'] = row['age'] + 1 # This works

print(df.head()) # Ages incremented by 1
# NOTE: Avoid iterrows for large DataFrames - it's slow!
```

Last executed at 2026-06-06 01:07:01 in 16ms

```
Using iterrows():
Index: 0, Name: Alice, Age: 25
Index: 1, Name: Bob, Age: 30
Index: 2, Name: Charlie, Age: 35
Index: 3, Name: Diana, Age: 28
Index: 4, Name: Eve, Age: 32
Index: 5, Name: Frank, Age: 27
```

	name	age	city	salary
0	Alice	26	NYC	50000
1	Bob	31	LA	60000
2	Charlie	36	NYC	70000
3	Diana	29	Miami	55000
4	Eve	33	LA	65000

Pandas

Select by condition (`df['age'] > 20`)

Query (`df.query('age > 28 and city == "NYC"')`)

`df.isna()`

`df.dropna()`

`df.fillna(0)`

`df.duplicated()`

`df.groupby('department')`

Joins (`pd.merge(employees, departments, on='dept_id', how='left')`)

Sort

Rank



Pandas

```
import pandas as pd

# CSV
df.to_csv('output.csv', index=False)
df_csv = pd.read_csv('file.csv')

# Excel
df.to_excel('output.xlsx', sheet_name='Data', index=False)
df_excel = pd.read_excel('file.xlsx', sheet_name='Data')

# JSON
df.to_json('output.json', orient='records', indent=2)
df_json = pd.read_json('file.json')

# Parquet (best for performance)
df.to_parquet('output.parquet')
df_parquet = pd.read_parquet('file.parquet')

# SQL
from sqlalchemy import create_engine
engine = create_engine('sqlite:///database.db')
df.to_sql('table_name', engine, if_exists='replace')
df_sql = pd.read_sql('SELECT * FROM table_name', engine)

# Clipboard (from Excel or web)
df_from_clipboard = pd.read_clipboard()
```





SQL

SQL

Query Query History

```
1 SELECT id, name, version FROM bdc.collections;
2
3 -- SELECT * FROM bdc.collections;
4
5 -- SELECT id, name, version FROM bdc.collections ORDER BY id DESC;
6
7 -- SELECT name, COUNT(*) FROM bdc.collections GROUP by name;
8
```

Data Output Messages Notifications



	id [PK] integer	name character varying (255)	version character varying
1	54	LANDSAT-6M-SP-IDT	1
2	106	S2-3M	1
3	28	CBERS-WFI-8D	1
4	11	landsat	2
5	18	landsat-3m-amazonia-1dt	1
6	72	LCC_L8_30_16D_STK_Cerrado	1
7	38	mosaic-cbers4a-paraiba-3m	1
8	10	S2V3_10	2
9	8	sentinel-3-olci-l1-bundle	1
10	56	mosaic-landsat-sp-6m	1
11	40	mosaic-s2-paraiba-3m	1
12	131	MODISA-OCSMART-POC-DAILY	1

SQL

Query Query History

Scratch Pad X

```
1 -- SELECT id, name, version FROM bdc.collections;
2
3 SELECT * FROM bdc.collections;
4
5 -- SELECT id, name, version FROM bdc.collections ORDER BY id DESC;
6
7 -- SELECT name, COUNT(*) FROM bdc.collections GROUP by name;
8
```

Data Output Messages Notifications

SQL

Showing rows: 1 to 129 Page No: 1 of 1

	id [PK] integer	name character varying (255)	title character varying (255)	description text	temporal_composition_schema jsonb	composite_function_id integer	grid_ref_sys_id integer	collection_type collection_type	metadat jsonb
1	54	LANDSAT-6M-SP-IDT	Landsat Image Identity ...	Earth Obser...	null	1	11	cube	{\"license
2	106	S2-3M	Sentinel-2 image Mosai...	Sentinel-2 i...	{\"step\": 3, \"unit\": \"month\", \"sche...	5	10	cube	{\"license
3	28	CBERS-WFI-8D	CBERS/WFI - Level-4-S...	Earth Obser...	{\"step\": 8, \"unit\": \"day\", \"cycle\": {\"...	5	12	cube	{\"license
4	11	landsat	Landsat Collection 2 - L...	Landsat Col...	null	[null]	2	collection	{\"license
5	18	landsat-3m-amazonia-idt	Landsat 3M - Amazonia	Earth Obser...	null	1	11	cube	{\"license
6	72	LCC_L8_30_16D_STK_Cerrado	LCC - Cerrado - LC8 30...	This is a lan...	null	[null]	5	classification	{\"license
7	38	mosaic-cbers4a-paraiba-3m	CBERS-4A/WFI 3M Par...	CBERS-4A/...	{\"step\": 3, \"unit\": \"month\", \"sche...	5	[null]	mosaic	{\"license
8	10	S2V3_10	Sentinel 2 V3		null	1	10	cube	{\"license
9	8	sentinel-3-olci-l1-bundle	Sentinel-3/OLCI - Level...	Copernicus ...	null	[null]	[null]	collection	{\"license
10	56	mosaic-landsat-sp-6m	Landsat 6M São Paulo ...	Earth Obser...	{\"step\": 6, \"unit\": \"month\", \"sche...	5	11	mosaic	{\"license
11	40	mosaic-s2-paraiba-3m	S2 3M Paraiba State M...	Sentinel-2 i...	{\"step\": 3, \"unit\": \"month\", \"sche...	5	[null]	mosaic	{\"license
12	121	MODIS_OCSMART_POC_DAILY	MODIS Aqua Daily Parti...	The MODIS	{\"step\": 1, \"unit\": \"day\", \"cycle\": {\"...	[null]	[null]	collection	{\"license

SQL

Query Query History

```
1  -- SELECT id, name, version FROM bdc.collections;
2
3  -- SELECT * FROM bdc.collections;
4
5  SELECT id, name, version FROM bdc.collections ORDER BY id DESC;
6
7  -- SELECT name, COUNT(*) FROM bdc.collections GROUP by name;
8
```

Data Output Messages Notifications



	id [PK] integer	name character varying (255)	version character varying
1	133	sentinel-2-s2cloudless	1
2	132	sentinel-2-fmask	5
3	131	MODISA-OCSMART-POC-DAILY	1
4	130	MODISA-OCSMART-POC-MONTHLY	1
5	129	MODISA-OCSMART-aCDOM-DAILY	1
6	128	MODISA-OCSMART-aCDOM-MONTHLY	1
7	127	s3-olci-rrs-water	1
8	126	s3-olci-pc-tiete	1
9	125	CB4A-MUX-L4-SR	1
10	124	MODISA-OCSMART-SPM-DAILY	1
11	123	MODISA-OCSMART-SPM-MONTHLY	1

SQL

Query Query History

```
1  -- SELECT id, name, version FROM bdc.collections;  
2  
3  -- SELECT * FROM bdc.collections;  
4  
5  -- SELECT id, name, version FROM bdc.collections ORDER BY id DESC;  
6  
7  SELECT name, COUNT(*) FROM bdc.collections GROUP by name;  
8
```

Data Output Messages Notifications



	name character varying (255)	count bigint
69	mod11a2	1
70	mod11a2_bundle	1
71	mod13q1	2
72	mod13q1_bundle	1
73	MODISA-OCSMART-aCDOM-DAILY	1
74	MODISA-OCSMART-aCDOM-MONTHLY	1
75	MODISA-OCSMART-CHL-DAILY	1
76	MODISA-OCSMART-CHL-MONTHLY	1
77	MODISA-OCSMART-POC-DAILY	1
78	MODISA-OCSMART-POC-MONTHLY	1
79	MODISA-OCSMART-RRS-DAILY	1

SQL

Query Query History

```
1  -- INSERT INTO students (name, age) VALUES ('John', 20);
2
3  UPDATE bdc.collections SET is_public=true WHERE id IN (128,129);
4
5  -- DELETE FROM bdc.items WHERE collection_id IN (128) AND name IN ("nome_do_item")
6
```

Query Query History

```
1  -- Create table
2  CREATE TABLE students (
3      id INT PRIMARY KEY,
4      name VARCHAR(100)
5  );
6
7  -- Alter structure
8  ALTER TABLE students ADD email VARCHAR(100);
9
10 -- Drop table
11 DROP TABLE students;
```

SQL

Query Query History

```
1  -- SELECT id, name, version FROM bdc.collections ORDER BY id ASC;
2
3  SELECT items.name, collections.name
4  FROM bdc.items
5  INNER JOIN bdc.collections ON collections.id = items.collection_id
6  WHERE collections.id = 4
7  LIMIT 10;
8
```

Data Output Messages Notifications

         SQL

	name character varying	 name character varying (255)
1	S2B_MSIL2A_20210819T140049_N0301_R067_T21LXG_20210822T104630	S2_L2A
2	S2B_MSIL2A_20210819T140049_N0301_R067_T21LXG_20210821T125850	S2_L2A
3	S2A_MSIL2A_20210824T140021_N0301_R067_T21LXG_20210824T182933	S2_L2A
4	S2B_MSIL2A_20210829T140019_N0301_R067_T21LXG_20210829T182903	S2_L2A
5	S2A_MSIL2A_20210903T140021_N0301_R067_T21LXG_20210903T182153	S2_L2A
6	S2B_MSIL2A_20210908T140019_N0301_R067_T21LXG_20210908T182410	S2_L2A
7	S2A_MSIL2A_20210913T140021_N0301_R067_T21LXG_20210913T165455	S2_L2A
8	S2B_MSIL2A_20210918T140019_N0301_R067_T21LXG_20210918T182705	S2_L2A
9	S2A_MSIL2A_20210923T140021_N0301_R067_T21LXG_20210923T164144	S2_L2A
10	S2B_MSIL2A_20210928T140019_N0301_R067_T21LXG_20210928T182840	S2_L2A

